

Feature Selection Prima della CV

Come una Procedura Innocente Invalida l'Intera Analisi

Francesco Faillace, Lorenzo Maria Pennelli, Eros Pinzani — Statistical Learning A.A. 2025/26

10 giugno 2026

Contents

1	Setup	2
1.1	Pacchetti	2
1.2	Parametri Globali	2
2	Background Teorico	3
2.1	Il Data Generating Process	3
2.2	Il Meccanismo del Leakage	3
2.3	Collegamento all'Ottimismo del Corso	4
2.4	Derivazione Teorica: perché l'ottimismo cresce come $\sqrt{2 \log p/n}$	4
3	Funzioni Principali	5
4	Esperimento 1 — Dimostrazione Base	10
4.1	Il Risultato Hero ($n = 200, p = 500, k = 20$)	10
4.2	Heatmap dell'Ottimismo $f(n, p)$ a $k = 20$	11
4.3	L'Effetto di k	13
5	Esperimento 2 — Validazione della Previsione Teorica	14
6	[APPENDICE — SLIDE DI BACKUP] Esperimento 3: Robustezza con Predittori Correlati	16
6.1	Struttura AR(1)	16
6.2	Struttura a Blocchi	17
7	Esperimento 4 — Metodi Embedded: Filter vs Ridge Sbagliato vs Ridge Corretto	19
7.1	Approfondimento: perché Ridge e non Lasso	19
8	Conclusioni	23
8.1	Tre messaggi da portare a casa	23
8.2	Questa non è teoria accademica	23
9	Riferimenti	24

1 Setup

1.1 Pacchetti

```
# Lista dei pacchetti necessari
pkgs <- c("glmnet", "ggplot2", "dplyr", "tidyr",
          "patchwork", "reshape2", "scales", "MASS")

# Installazione automatica solo dei pacchetti mancanti
to_install <- pkgs[!pkgs %in% installed.packages()[, "Package"]]
if (length(to_install) > 0) {
  repos <- getOption("repos")
  if (is.null(repos) || identical(repos[["CRAN"]], "@CRAN@") || is.na(repos[["CRAN"]])) {
    repos[["CRAN"]] <- "https://cloud.r-project.org"
    options(repos = repos)
  }
  install.packages(to_install, dependencies = TRUE, quiet = TRUE)
}

suppressPackageStartupMessages({
  library(glmnet)      # Lasso e Elastic Net
  library(ggplot2)     # visualizzazioni
  library(dplyr)       # manipolazione dati
  library(tidyr)       # pivoting e reshaping
  library(patchwork)   # composizione di grafici multipli
  library(reshape2)    # melt per le heatmap
  library(scales)      # scala colori nei grafici
  library(MASS)        # mvrnorm per predittori correlati
})

theme_set(theme_bw(base_size = 11))
```

1.2 Parametri Globali

```
# =====
# QUICK_RUN = TRUE → risultati rapidi (pochi minuti)
#                               indicato per sviluppo e test
# QUICK_RUN = FALSE → analisi completa (15-30 minuti)
#                               usare prima della presentazione finale
# =====
QUICK_RUN <- TRUE

# Numero di simulazioni Monte Carlo
B <- if (QUICK_RUN) 30 else 100
# Fold per la cross-validation esterna
FOLDS <- 10
# Fold per la CV interna del Ridge (Esperimento 4)
FOLDS_INNER <- 5
# Seed master per riproducibilità (seed per la b-esima run = SEED + b)
SEED <- 2025

# Griglia di parametri per la heatmap (Esperimento 1) e la
# validazione teorica (Esperimento 2)
```

```

if (QUICK_RUN) {
  grid_n <- c(50, 200, 500)
  grid_p <- c(20, 100, 500)
} else {
  grid_n <- c(50, 100, 200, 500)
  grid_p <- c(20, 50, 100, 200, 500)
}

cat("Configurazione:", if (QUICK_RUN) "QUICK (B =", "FULL (B =", B,
  "| grid n:", paste(grid_n, collapse = ","),
  "| grid p:", paste(grid_p, collapse = ","), ")\n")

## Configurazione: QUICK (B = FULL (B = 30 | grid n: 50,200,500 | grid p: 20,100,500 )

```

2 Background Teorico

2.1 Il Data Generating Process

In **tutti** gli esperimenti di questo notebook adottiamo un DGP in cui non esiste **nessun segnale predittivo**:

$$X_j \stackrel{\text{iid}}{\sim} \mathcal{N}(0, 1), \quad j = 1, \dots, p$$

$$Y \sim \text{Bernoulli}(0.5), \quad Y \perp X_1, \dots, X_p$$

Qualsiasi AUC significativamente superiore a 0.5 non riflette una capacità predittiva reale: è un **artefatto della procedura di analisi**.

Nota sul DGP: Y è binaria perché usiamo l'AUC come metrica di valutazione, che è definita per classificatori. Un DGP con Y continua e RMSE come metrica produrrebbe lo stesso fenomeno, ma l'AUC è più drammaticamente comunicabile — un valore di 0.85 in un contesto di classificazione è inequivocabilmente interpretato come “il modello funziona bene”.

2.2 Il Meccanismo del Leakage

La cross-validation simula il caso in cui un modello viene addestrato su certi dati e valutato su dati **mai visti**. L'ipotesi fondamentale è che il validation set sia completamente ignoto durante la fase di training.

Quando si selezionano le variabili **prima** del loop di CV, questa ipotesi viene violata silenziosamente:

La selezione delle top- k variabili usa la correlazione di ogni X_j con Y calcolata sull'**intero dataset** — incluse le osservazioni che finiranno nel validation set.

Le variabili selezionate non sono quelle davvero correlate con Y nella popolazione (non ce ne sono, per costruzione), ma quelle che per **puro caso** mostrano la correlazione campionaria più alta con Y in quel campione specifico. La CV valuta il modello come se queste variabili fossero scelte in modo “onesto”, ma non lo sono.

Pipeline errata:

1. Calcola $\text{cor}(X_j, Y)$ per tutti i p predittori sull'INTERO dataset
2. Seleziona i top- k per valore assoluto della correlazione
3. Applica 10-fold CV sul modello con i k predittori selezionati ← troppo tardi

Pipeline corretta:

Per ogni fold $f = 1, \dots, 10$:

1. Usa SOLO il training fold per calcolare $\text{cor}(X_j, Y_{\text{train}})$
2. Seleziona i top-k ← la selezione non vede mai il validation set
3. Fitta il modello sul training fold con i k predittori selezionati
4. Valuta sul validation fold f

2.3 Collegamento all'Ottimismo del Corso

Il termine di ottimismo dell'errore di training, derivato a lezione, è:

$$\text{Opt} = \frac{2}{n} \sum_{i=1}^n \text{Cov}(Y_i, \hat{Y}_i)$$

Questo termine misura quanto le predizioni $\hat{Y}_i$ sono artificialmente vicine alle Y_i su cui il modello viene poi valutato. Nella pipeline corretta, il modello non ha mai visto le Y_i del validation set, quindi la covarianza per quelle osservazioni è piccola.

Nella pipeline errata, le k variabili selezionate sono state scelte **perché** covariavano con Y sull'intero dataset. I fitted values $\hat{Y}_i = \hat{\beta}^\top x_i$ sono costruiti su predittori già “ottimizzati” rispetto alle stesse Y_i su cui vengono valutati:

$$\text{Cov}(Y_i, \hat{Y}_i) \Big|_{\text{pipeline errata}} \gg \text{Cov}(Y_i, \hat{Y}_i) \Big|_{\text{pipeline corretta}}$$

Il termine di ottimismo viene inflazionato non dall'overfitting del modello, ma dalla **pre-selezione delle variabili**. La CV non può vederlo perché è avvenuto prima del loop.

2.4 Derivazione Teorica: perché l'ottimismo cresce come $\sqrt{2 \log p / n}$

Questa sezione costruisce la previsione teorica che verrà poi confrontata con le simulazioni nell'Esperimento 2.

Passo 1 — Distribuzione della correlazione campionaria sotto H_0 .

Definiamo la correlazione campionaria punto-biserial tra X_j e Y :

$$r_j = \frac{\sum_{i=1}^n (x_{ij} - \bar{x}_j)(y_i - \bar{y})}{(n-1) s_j s_y}$$

Nota tecnica: con Y binaria e X_j continua, questa è formalmente la correlazione punto-biserial, che coincide algebricamente con la correlazione di Pearson ed ha la medesima struttura asintotica sotto indipendenza.

Sotto H_0 : $X_j \perp Y$, per il Teorema Centrale del Limite applicato al prodotto di due variabili indipendenti a media zero:

$$\sqrt{n} r_j \xrightarrow{d} \mathcal{N}(0, 1)$$

ovvero $r_j \approx \mathcal{N}(0, 1/n)$ per n sufficientemente grande.

Passo 2 — Distribuzione del massimo di p correlazioni.

Se $r_1, \dots, r_p$ sono approssimativamente i.i.d. $\mathcal{N}(0, 1/n)$, i valori standardizzati $Z_j = \sqrt{n} r_j$ sono approssimativamente i.i.d. $\mathcal{N}(0, 1)$.

Per un risultato classico sulle statistiche d'ordine delle normali standard, il massimo in valore assoluto di p variabili i.i.d. $\mathcal{N}(0, 1)$ si concentra attorno a:

$$E\left[\max_{j=1}^p |Z_j|\right] \approx \sqrt{2 \log p}$$

Derivazione informale: cerchiamo t^* tale che $P(\max_j |Z_j| > t^*) \approx \frac{1}{2}$. Applicando l'union bound e l'approssimazione di Mills $1 - \Phi(t) \approx \phi(t)/t$ per t grande:

$$p \cdot 2(1 - \Phi(t^*)) \approx \frac{1}{2} \implies \frac{2p \phi(t^*)}{t^*} \approx \frac{1}{2} \implies t^{*2} \approx 2 \log p$$

Passo 3 — Previsione dell'ottimismo.

Riportando alla scala originale delle correlazioni (dividendo per $\sqrt{n}$):

$$\max_{j=1}^p |r_j| \approx \sqrt{\frac{2 \log p}{n}}$$

La variabile “migliore” tra p candidate, per puro caso, ha correlazione campionaria con Y pari a circa $\sqrt{2 \log p / n}$. Selezionando le top- k su questa base e costruendo un classificatore, l'ottimismo in AUC è proporzionale a questo valore.

Esempio numerico: per $n = 200$, $p = 500$, $k = 20$:

$$\sqrt{\frac{2 \log 500}{200}} \approx \sqrt{\frac{2 \times 6.21}{200}} \approx \sqrt{0.062} \approx 0.25$$

Una correlazione campionaria di 0.25 su puro rumore è sufficiente per portare l'AUC stimato dalla CV dal suo valore teorico di 0.50 a valori vicini a 0.85.

3 Funzioni Principali

```
# =====
# FUNZIONE 1: Calcolo dell'AUC (metodo rank-based / Wilcoxon)
#
# Equivalente alla statistica U di Wilcoxon-Mann-Whitney:
# P(score di un positivo > score di un negativo)
# Non richiede pacchetti esterni ed è numericamente stabile.
# =====
compute_auc <- function(y_true, y_prob) {
  y_true <- as.integer(y_true)
  n1 <- sum(y_true == 1)
  n0 <- sum(y_true == 0)
  # Casi degeneri: se un solo fold non ha positivi o negativi
  if (n1 == 0 || n0 == 0) return(NA_real_)
  r <- rank(y_prob, ties.method = "average")
  (sum(r[y_true == 1]) - n1 * (n1 + 1) / 2) / (n1 * n0)
}

# =====
# FUNZIONE 2: CV con pipeline ERRATA (leakage)
#
# La feature selection avviene FUORI dal loop di CV:
```

```

# le correlazioni vengono calcolate sull'intero dataset,
# incluse le osservazioni che finiranno nel validation set.
# Questo è il difetto strutturale che produce AUC inflazionato.
# =====
cv_auc_leaky <- function(X, y, k, n_folds = 10) {
  n <- nrow(X)
  # Assegna nomi espliciti alle colonne: quando R converte una
  # matrice indicizzata in data.frame usa il deparse dell'espressione
  # come prefisso (es. "X_sel.tr..." vs "X_sel.te..."), producendo
  # nomi diversi tra train e test e rompendo predict().
  # Assegnare nomi fissi risolve il problema alla radice.
  colnames(X) <- paste0("x", seq_len(ncol(X)))

  # === BUG: selezione delle top-k usando TUTTO il dataset ===
  cors <- abs(cor(X, y)) # correlazione punto-biserial
  top_k <- order(cors, decreasing = TRUE)[1:k]
  X_sel <- X[, top_k, drop = FALSE] # sottomatrice con le k variabili "migliori"
  # =====

  # CV sul modello con le variabili già selezionate
  folds <- sample(rep(seq_len(n_folds), length.out = n))
  aucs <- numeric(n_folds)

  for (f in seq_len(n_folds)) {
    tr_idx <- which(folds != f)
    te_idx <- which(folds == f)

    df_tr <- data.frame(y = y[tr_idx], X_sel[tr_idx, , drop = FALSE])
    df_te <- as.data.frame(X_sel[te_idx, , drop = FALSE])

    fit <- tryCatch(
      glm(y ~ ., data = df_tr, family = binomial()),
      error = function(e) NULL
    )
    if (is.null(fit)) { aucs[f] <- 0.5; next }

    probs <- predict(fit, newdata = df_te, type = "response")
    aucs[f] <- compute_auc(y[te_idx], probs)
  }
  mean(aucs, na.rm = TRUE)
}

# =====
# FUNZIONE 3: CV con pipeline CORRETTA
#
# La feature selection avviene DENTRO ogni fold:
# le correlazioni vengono calcolate solo sul training fold,
# senza mai "vedere" le osservazioni di validazione.
# =====
cv_auc_correct <- function(X, y, k, n_folds = 10) {
  n <- nrow(X)
  # Stessa ragione di cv_auc_leaky: nomi espliciti per coerenza
  # tra i data.frame di training e test in ogni fold.

```

```

colnames(X) <- paste0("x", seq_len(ncol(X)))
folds <- sample(rep(seq_len(n_folds), length.out = n))
aucs <- numeric(n_folds)

for (f in seq_len(n_folds)) {
  tr_idx <- which(folds != f)
  te_idx <- which(folds == f)

  # === CORRETTO: selezione solo sul training fold ===
  cors <- abs(cor(X[tr_idx, ], y[tr_idx]))
  top_k <- order(cors, decreasing = TRUE)[1:k]
  # =====

  X_tr_sel <- X[tr_idx, top_k, drop = FALSE]
  X_te_sel <- X[te_idx, top_k, drop = FALSE]

  df_tr <- data.frame(y = y[tr_idx], X_tr_sel)
  df_te <- as.data.frame(X_te_sel)

  fit <- tryCatch(
    glm(y ~ ., data = df_tr, family = binomial()),
    error = function(e) NULL
  )
  if (is.null(fit)) { aucs[f] <- 0.5; next }

  probs <- predict(fit, newdata = df_te, type = "response")
  aucs[f] <- compute_auc(y[te_idx], probs)
}
mean(aucs, na.rm = TRUE)
}

# =====
# FUNZIONE 4: CV Ridge con pipeline ERRATA
#
# Usiamo Ridge (alpha=0) invece di Lasso (alpha=1) per una
# ragione di chiarezza visiva: con dati puro rumore il Lasso
# tende ad azzerare quasi tutti i coefficienti in molte run,
# producendo AUC 0.5 anche nella pipeline sbagliata e
# rendendo invisibile il leakage nel boxplot.
# Ridge mantiene attivi TUTTI i p=500 predittori (compressi
# ma non annullati), accumulando il segnale spurio di ciascuno
# in modo consistente → leakage stabile e visibile in ogni run.
#
# Il vantaggio concettuale aggiuntivo: Ridge non fa selezione
# di variabili, quindi la demo diventa ancora più forte -
# anche un metodo che non seleziona nulla soffre dello stesso
# leakage se lambda viene tuned fuori dal loop.
#
# Differenza cruciale rispetto al "filter sbagliato":
# l'errore non è nel tipo di penalizzazione, ma nel fatto che
# l'iperparametro lambda è stato ottimizzato guardando Y
# su tutto il dataset PRIMA della CV di valutazione.
# =====

```

```

ridge_auc_wrong <- function(X, y, n_folds = 10) {
  n <- nrow(X)

  # === BUG: tuning di lambda sull'intero dataset ===
  cv_full <- cv.glmnet(X, y, family = "binomial",
                      type.measure = "auc",
                      nfolds = n_folds, alpha = 0) # alpha=0: Ridge
  lambda_contam <- cv_full$lambda.min # lambda "contaminato"
  # =====

  # CV separata con lambda fissato (il danno è già fatto)
  folds <- sample(rep(seq_len(n_folds), length.out = n))
  aucs <- numeric(n_folds)

  for (f in seq_len(n_folds)) {
    tr_idx <- which(folds != f)
    te_idx <- which(folds == f)

    fit <- glmnet(X[tr_idx, ], y[tr_idx], family = "binomial",
                 alpha = 0, lambda = lambda_contam) # alpha=0: Ridge
    probs <- as.numeric(predict(fit, X[te_idx, ], type = "response"))
    aucs[f] <- compute_auc(y[te_idx], probs)
  }
  mean(aucs, na.rm = TRUE)
}

# =====
# FUNZIONE 5: CV Ridge con pipeline CORRETTA (nested CV)
#
# Per ogni fold esterno:
# 1. cv.glmnet(alpha=0) sul TRAINING fold → lambda ottimale
# 2. Fit Ridge con quel lambda sul training fold
# 3. Valutazione sul validation fold esterno
#
# Lambda è scelto senza mai vedere le osservazioni di test.
# =====
ridge_auc_correct <- function(X, y, n_folds_outer = 10,
                             n_folds_inner = 5) {
  n <- nrow(X)
  folds <- sample(rep(seq_len(n_folds_outer), length.out = n))
  aucs <- numeric(n_folds_outer)

  for (f in seq_len(n_folds_outer)) {
    tr_idx <- which(folds != f)
    te_idx <- which(folds == f)

    # === CORRETTO: tuning di lambda solo sul training fold ===
    cv_tr <- tryCatch(
      cv.glmnet(X[tr_idx, ], y[tr_idx], family = "binomial",
                type.measure = "auc",
                nfolds = n_folds_inner, alpha = 0), # alpha=0: Ridge
      error = function(e) NULL
    )
  }
}

```



```

if (is.null(cv_tr)) { aucs[f] <- 0.5; next }
lambda_ok <- cv_tr$lambda.min
# =====

fit <- glmnet(X[tr_idx, ], y[tr_idx], family = "binomial",
              alpha = 0, lambda = lambda_ok) # alpha=0: Ridge
probs <- as.numeric(predict(fit, X[te_idx, ], type = "response"))
aucs[f] <- compute_auc(y[te_idx], probs)
}
mean(aucs, na.rm = TRUE)
}

# =====
# GENERATORI DI DATI
# =====

# Predittori i.i.d.  $N(0,1)$ : baseline degli Esperimenti 1-4
gen_iid <- function(n, p) matrix(rnorm(n * p), n, p)

# Predittori con struttura AR(1): correlazione  $\rho^{|i-j|}$ 
# Simulazione sequenziale efficiente:  $X[j] = \rho X[j-1] + \text{eps}$ 
gen_ar1 <- function(n, p, rho) {
  X <- matrix(0, n, p)
  X[, 1] <- rnorm(n)
  sd_innov <- sqrt(1 - rho^2) # varianza dell'innovazione
  for (j in 2:p) {
    X[, j] <- rho * X[, j - 1] + rnorm(n, sd = sd_innov)
  }
  X
}

# Predittori a struttura a blocchi:
# all'interno di ogni blocco  $\text{cor}(X_i, X_j) = \rho_{\text{within}}$ 
# tra blocchi diversi: indipendenti
gen_blocks <- function(n, p, n_blocks = 10, rho_within = 0.8) {
  stopifnot(p %% n_blocks == 0)
  bsize <- p %% n_blocks
  X <- matrix(0, n, p)
  for (b in seq_len(n_blocks)) {
    idx <- ((b - 1) * bsize + 1):(b * bsize)
    fac <- rnorm(n) # fattore comune al blocco
    for (j in idx) {
      # Decompone in componente comune + rumore idiosincratico
      X[, j] <- sqrt(rho_within) * fac + sqrt(1 - rho_within) * rnorm(n)
    }
  }
  X
}

```

4 Esperimento 1 — Dimostrazione Base

Questo esperimento risponde alla domanda fondamentale: *quanto è grande il leakage in uno scenario realistico?*

Fissa i parametri al caso di riferimento ($n = 200$, $p = 500$, $k = 20$) e misura l'AUC stimato dalla CV con pipeline errata e corretta su 30 run Monte Carlo. La terza dimensione di analisi è la heatmap che mostra come l'ottimismo varia al variare di n e p a $k = 20$ fisso.

4.1 Il Risultato Hero ($n = 200$, $p = 500$, $k = 20$)

```
# Parametri del caso di riferimento
n_hero <- 200; p_hero <- 500; k_hero <- 20

# Monte Carlo: B run con seed diversi per riproducibilità
res_hero <- matrix(NA, B, 2, dimnames = list(NULL, c("leaky", "correct")))

for (b in seq_len(B)) {
  set.seed(SEED + b)
  X <- gen_iid(n_hero, p_hero)
  y <- rbinom(n_hero, 1, 0.5)      # Y ~ Bernoulli(0.5), indep. da X

  res_hero[b, "leaky"] <- cv_auc_leaky(X, y, k = k_hero, n_folds = FOLDS)
  res_hero[b, "correct"] <- cv_auc_correct(X, y, k = k_hero, n_folds = FOLDS)

  if (b %% 10 == 0) cat("  Run", b, "/", B, "\n")
}

##   Run 10 / 30
##   Run 20 / 30
##   Run 30 / 30

# Statistiche riassuntive
cat("\n--- Risultati Hero (n=200, p=500, k=20, B=", B, "run) ---\n")

##
## --- Risultati Hero (n=200, p=500, k=20, B= 30 run) ---
cat("AUC pipeline ERRATA : media =", round(mean(res_hero[, "leaky"]), 3),
    " | sd =", round(sd(res_hero[, "leaky"]), 3), "\n")

## AUC pipeline ERRATA : media = 0.811 | sd = 0.027
cat("AUC pipeline CORRETTA: media =", round(mean(res_hero[, "correct"]), 3),
    " | sd =", round(sd(res_hero[, "correct"]), 3), "\n")

## AUC pipeline CORRETTA: media = 0.512 | sd = 0.063
cat("Ottimismo medio (AUC_errata - AUC_corretta):",
    round(mean(res_hero[, "leaky"]) - mean(res_hero[, "correct"]), 3), "\n")

## Ottimismo medio (AUC_errata - AUC_corretta): 0.298

# Preparazione dati in formato long per ggplot2
df_hero <- data.frame(res_hero) |>
  pivot_longer(everything(), names_to = "pipeline", values_to = "auc") |>
  mutate(pipeline = factor(pipeline,
    levels = c("leaky", "correct"),
```

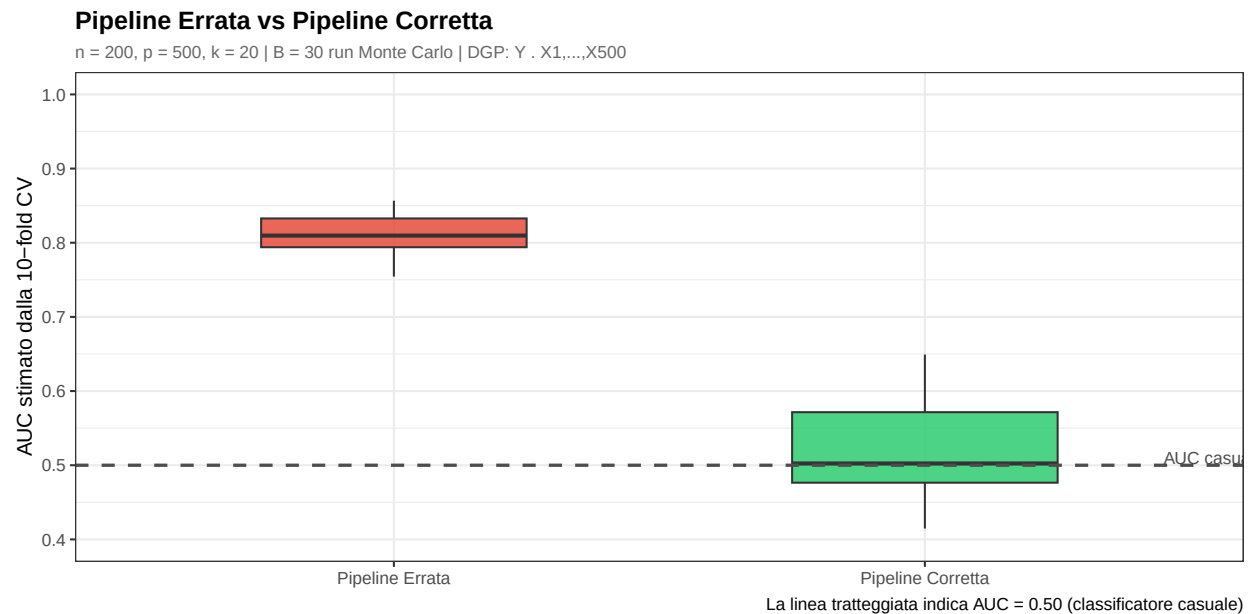
```

labels = c("Pipeline Errata", "Pipeline Corretta"))

# Boxplot affiancato - la "hero slide" del progetto
p_hero_plot <- ggplot(df_hero, aes(x = pipeline, y = auc, fill = pipeline)) +
  geom_boxplot(width = 0.5, outlier.size = 1.5, alpha = 0.85) +
  geom_hline(yintercept = 0.5, linetype = "dashed", linewidth = 0.8,
    color = "grey30") +
  annotate("text", x = 2.45, y = 0.51, label = "AUC casuale = 0.50",
    size = 3.2, color = "grey30", hjust = 0) +
  scale_fill_manual(values = c("Pipeline Errata" = "#e74c3c",
    "Pipeline Corretta" = "#2ecc71")) +
  scale_y_continuous(limits = c(0.40, 1.00), breaks = seq(0.4, 1.0, 0.1)) +
  labs(
    title = "Pipeline Errata vs Pipeline Corretta",
    subtitle = paste0("n = 200, p = 500, k = 20 | B = ", B,
      " run Monte Carlo | DGP: Y ~ X1,...,X500"),
    x = NULL, y = "AUC stimato dalla 10-fold CV",
    caption = "La linea tratteggiata indica AUC = 0.50 (classificatore casuale)"
  ) +
  theme(legend.position = "none",
    plot.title = element_text(face = "bold"),
    plot.subtitle = element_text(size = 9, color = "grey40"))

print(p_hero_plot)

```



La figura mostra il risultato centrale del progetto: le due distribuzioni di AUC **non si sovrappongono quasi per niente**, nonostante i dati siano puro rumore per costruzione.

4.2 Heatmap dell'Ottimismo $f(n, p)$ a $k = 20$

```

# Griglia completa: per ogni (n, p) calcola l'ottimismo medio su B run
# L'ottimismo è definito come AUC_errata - 0.50
# (usiamo 0.50 come baseline teorica invece di AUC_correct per

```

```

# separare l'effetto del leakage dalla varianza del CV)
grid_params <- expand_grid(n = grid_n, p = grid_p)
grid_params$ottimismo <- NA

for (i in seq_len(nrow(grid_params))) {
  ni <- grid_params$n[i]; pi <- grid_params$p[i]
  auc_vals <- numeric(B)
  for (b in seq_len(B)) {
    set.seed(SEED + b * 1000 + i)
    X <- gen_iid(ni, pi)
    y <- rbinom(ni, 1, 0.5)
    auc_vals[b] <- cv_auc_leaky(X, y, k = 20, n_folds = FOLDS)
  }
  grid_params$ottimismo[i] <- mean(auc_vals, na.rm = TRUE) - 0.50
  cat("  n =", ni, "p =", pi,
      "| ottimismo medio =", round(grid_params$ottimismo[i], 3), "\n")
}

```

```

##  n = 50 p = 20 | ottimismo medio = 0.011
##  n = 200 p = 20 | ottimismo medio = 0.001
##  n = 500 p = 20 | ottimismo medio = -0.002

##  n = 50 p = 100 | ottimismo medio = 0.224
##  n = 200 p = 100 | ottimismo medio = 0.194
##  n = 500 p = 100 | ottimismo medio = 0.138

##  n = 50 p = 500 | ottimismo medio = 0.368
##  n = 200 p = 500 | ottimismo medio = 0.313
##  n = 500 p = 500 | ottimismo medio = 0.215

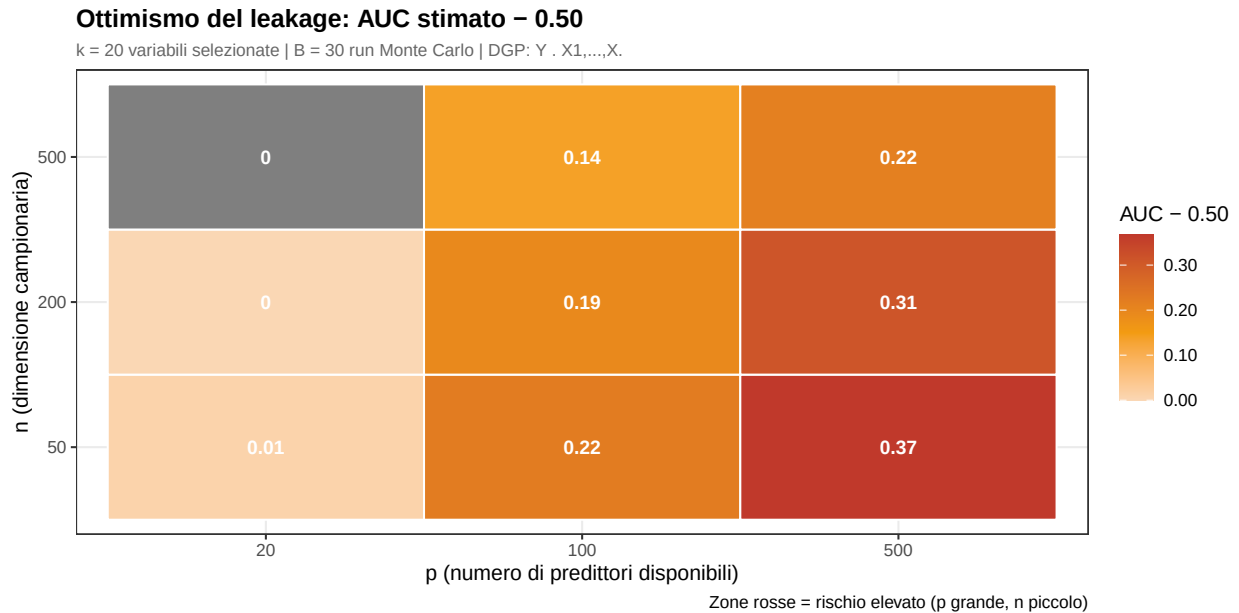
```

```

p_heat <- ggplot(grid_params,
  aes(x = factor(p), y = factor(n), fill = ottimismo)) +
  geom_tile(color = "white", linewidth = 0.5) +
  geom_text(aes(label = round(ottimismo, 2)), size = 3.5,
    color = "white", fontface = "bold") +
  scale_fill_gradient2(
    low = "#f0f4f8",
    mid = "#f39c12",
    high = "#c0392b",
    midpoint = 0.15,
    name = "AUC - 0.50",
    limits = c(0, NA),
    labels = scales::number_format(accuracy = 0.01)
  ) +
  labs(
    title = "Ottimismo del leakage: AUC stimato - 0.50",
    subtitle = paste0("k = 20 variabili selezionate | B = ", B,
      " run Monte Carlo | DGP: Y  X ,...,X"),
    x = "p (numero di predittori disponibili)",
    y = "n (dimensione campionaria)",
    caption = "Zone rosse = rischio elevato (p grande, n piccolo)"
  ) +
  theme(plot.title = element_text(face = "bold"),
    plot.subtitle = element_text(size = 9, color = "grey40"))

```

```
print(p_heat)
```



La heatmap mostra come il rischio sia concentrato nell'angolo **alto-p / basso-n**: con molti predittori e pochi dati, la procedura errata produce stime completamente fasulle. Le coordinate di dataset biomedici comuni (es. microarray: $n \approx 50$, $p \approx 1000$) cadono sistematicamente nella zona rossa.

4.3 L'Effetto di k

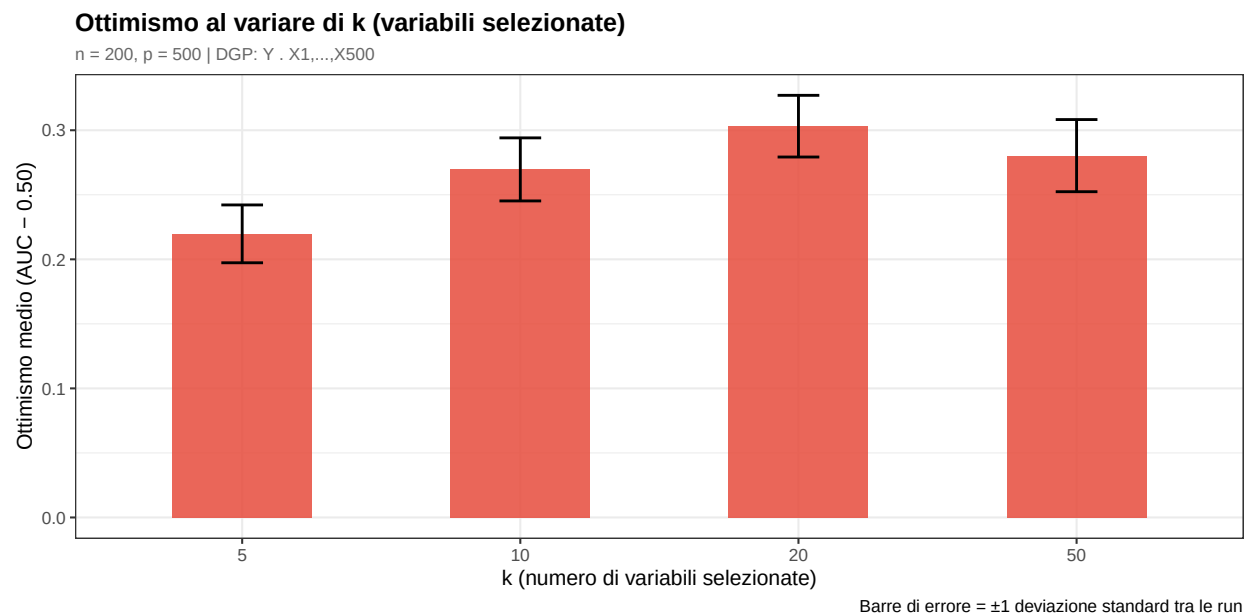
```
# A parità di n=200, p=500, come cambia l'ottimismo al variare di k?
k_vals <- c(5, 10, 20, 50)
res_k <- data.frame(k = k_vals, ottimismo = NA, sd = NA)
```

```
for (i in seq_along(k_vals)) {
  ki <- k_vals[i]
  aucs_k <- numeric(B)
  for (b in seq_len(B)) {
    set.seed(SEED + b + i * 10000)
    X <- gen_iid(200, 500)
    y <- rbinom(200, 1, 0.5)
    aucs_k[b] <- cv_auc_leaky(X, y, k = ki, n_folds = FOLDS)
  }
  res_k$ottimismo[i] <- mean(aucs_k, na.rm = TRUE) - 0.50
  res_k$sd[i] <- sd(aucs_k, na.rm = TRUE)
  cat(" k =", ki, "| AUC medio =",
      round(mean(aucs_k, na.rm = TRUE), 3),
      "| ottimismo =", round(res_k$ottimismo[i], 3), "\n")
}
```

```
## k = 5 | AUC medio = 0.72 | ottimismo = 0.22
## k = 10 | AUC medio = 0.77 | ottimismo = 0.27
## k = 20 | AUC medio = 0.803 | ottimismo = 0.303
## k = 50 | AUC medio = 0.78 | ottimismo = 0.28
```

```
p_k <- ggplot(res_k, aes(x = factor(k), y = ottimismo)) +
  geom_col(fill = "#e74c3c", alpha = 0.85, width = 0.5) +
  geom_errorbar(aes(ymin = ottimismo - sd, ymax = ottimismo + sd),
    width = 0.15, linewidth = 0.7) +
  labs(
    title = "Ottimismo al variare di k (variabili selezionate)",
    subtitle = "n = 200, p = 500 | DGP: Y ~ X_1, ..., X_p",
    x = "k (numero di variabili selezionate)",
    y = "Ottimismo medio (AUC - 0.50)",
    caption = "Barre di errore = ±1 deviazione standard tra le run"
  ) +
  theme(plot.title = element_text(face = "bold"),
    plot.subtitle = element_text(size = 9, color = "grey40"))

print(p_k)
```



All'aumentare di k l'ottimismo cresce: selezionando più variabili rumore, si accumula più “segnale spurio” — un effetto analogo al multiple testing.

5 Esperimento 2 — Validazione della Previsione Teorica

La Sezione 1.4 ha derivato che il massimo della correlazione campionaria sotto H_0 cresce come $\sqrt{2 \log p / n}$. Qui verifichiamo se questa previsione teorica **predice quantitativamente** l'ottimismo osservato nelle simulazioni.

```
# Usa i risultati già calcolati nella griglia dell'Esperimento 1
# Calcola il predittore teorico per ogni (n, p)
theory_df <- grid_params |>
  mutate(
    # Predittore teorico dalla derivazione della sezione 1.4
    teorico = sqrt(2 * log(p) / n),
    # Etichetta per il grafico
```

```

    label = paste0("(", n, ",", p, ")")
  )

# Fit di un modello lineare: ottimismo ~ a * teorico
# (senza intercetta: quando teorico=0 l'ottimismo deve essere 0)
lm_fit <- lm(ottimismo ~ 0 + teorico, data = theory_df)
r_sq    <- cor(theory_df$teorico, theory_df$ottimismo)^2

cat("--- Validazione Teorica ---\n")

## --- Validazione Teorica ---
cat("Coefficiente di calibrazione (a):", round(coef(lm_fit), 3), "\n")

## Coefficiente di calibrazione (a): 0.611
cat("R² (var. teorica → ottimismo sim.):", round(r_sq, 3), "\n")

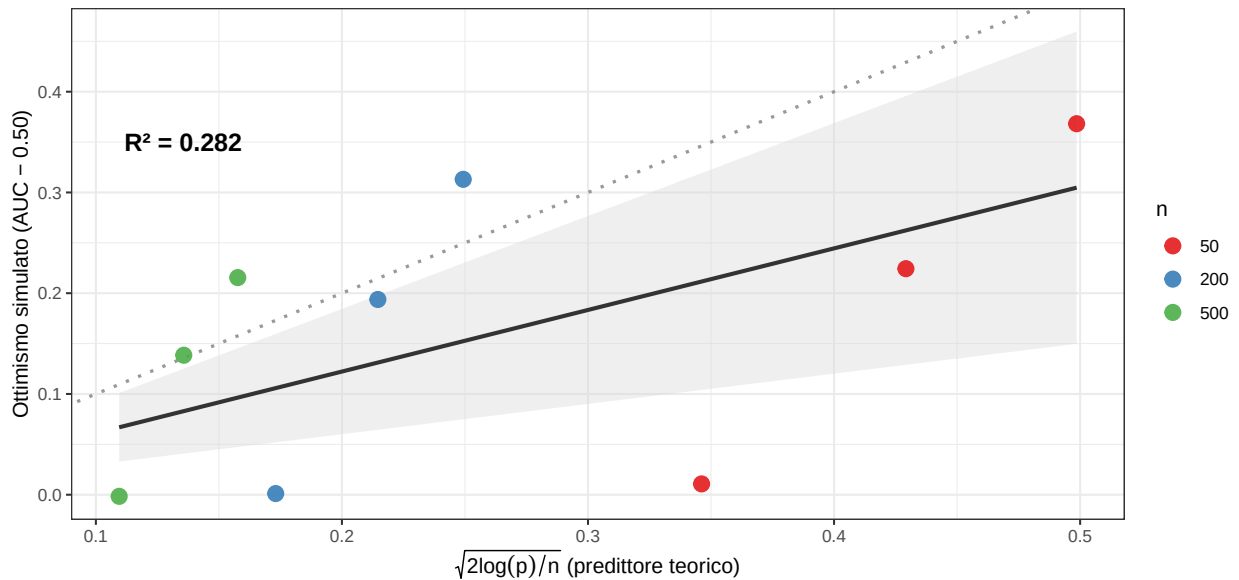
## R² (var. teorica → ottimismo sim.): 0.282
# Punti simulati vs curva teorica
p_theory <- ggplot(theory_df,
  aes(x = teorico, y = ottimismo, color = factor(n))) +
  geom_abline(slope = 1, intercept = 0, linetype = "dotted",
    color = "grey60", linewidth = 0.8) +
  geom_smooth(method = "lm", formula = y ~ 0 + x,
    se = TRUE, color = "grey20", fill = "grey85",
    linewidth = 1, inherit.aes = FALSE,
    aes(x = teorico, y = ottimismo)) +
  geom_point(size = 3.5, alpha = 0.9) +
  scale_color_brewer(palette = "Set1", name = "n") +
  annotate("text", x = min(theory_df$teorico) * 1.02,
    y = max(theory_df$ottimismo) * 0.95,
    label = paste0("R² = ", round(r_sq, 3)),
    size = 4.5, fontface = "bold", hjust = 0) +
  labs(
    title = "Simulazione vs Previsione Teorica",
    subtitle = bquote("Predittore teorico:" ~ sqrt(2*log(p)/n) ~
      "| k = 20 | B = " * .(B) * " run per cella"),
    x = expression(sqrt(2 * log(p) / n) ~ "(predittore teorico)"),
    y = "Ottimismo simulato (AUC - 0.50)",
    caption = paste0(
      "La linea tratteggiata è y=x (predizione perfetta); ",
      "la linea piena è la retta di calibrazione stimata (senza intercetta)"
    )
  ) +
  theme(plot.title = element_text(face = "bold"),
    plot.subtitle = element_text(size = 9, color = "grey40"))

print(p_theory)

```

Simulazione vs Previsione Teorica

Predittore teorico: $\sqrt{2\log(p)/n}$ | $k = 20$ | $B = 30$ run per cella



Un R^2 vicino a 1 su questo grafico indica che la formula $\sqrt{2\log p/n}$ non è solo una previsione qualitativa: **predice quantitativamente** l'ottimismo osservato nelle simulazioni, collegando il risultato empirico a un fondamento matematico preciso.

6 [APPENDICE — SLIDE DI BACKUP] Esperimento 3: Robustezza con Predittori Correlati

NOTA PER LA PRESENTAZIONE: Questo esperimento è stato escluso dalle slide principali per ragioni di tempo (10 minuti non sono sufficienti per 4 esperimenti + derivazione teorica). Le analisi e le tabelle qui presenti devono essere tenute **in appendice** come slide di backup, da mostrare **solo se la professoressa fa domande** sulla robustezza del risultato con predittori correlati.

Il messaggio verbale nella slide delle conclusioni è: *“Abbiamo verificato che la correlazione tra predittori peggiora il leakage, non lo attenua”* — con questo esperimento a supporto se richiesto.

Tutti gli esperimenti precedenti usano predittori i.i.d. In pratica, i predittori sono quasi sempre correlati tra loro. Questo esperimento verifica se la correlazione interna a X modifica il fenomeno del leakage.

Ipotesi da testare: con predittori correlati, la feature selection tende a scegliere *interi blocchi* di variabili simili anziché una sola, amplificando l'ottimismo rispetto al caso indipendente.

6.1 Struttura AR(1)

```
# Parametri fissi: n=200, p=100 (p ridotto per contenere il tempo con MASS::murnorm)
# k=10, rho variabile in {0, 0.3, 0.6, 0.9}
n_e3 <- 200; p_e3 <- 100; k_e3 <- 10
rho_vals <- c(0, 0.3, 0.6, 0.9)

res_ar1 <- data.frame(rho = rho_vals, auc_leaky = NA, auc_correct = NA)
```



```

for (i in seq_along(rho_vals)) {
  rho_i      <- rho_vals[i]
  auc_l <- auc_c <- numeric(B)

  for (b in seq_len(B)) {
    set.seed(SEED + b + i * 1e5)
    X <- if (rho_i == 0) gen_iid(n_e3, p_e3) else gen_ar1(n_e3, p_e3, rho_i)
    y <- rbinom(n_e3, 1, 0.5)

    auc_l[b] <- cv_auc_leaky(X, y, k = k_e3, n_folds = FOLDS)
    auc_c[b] <- cv_auc_correct(X, y, k = k_e3, n_folds = FOLDS)
  }
  res_ar1$auc_leaky[i] <- mean(auc_l, na.rm = TRUE)
  res_ar1$auc_correct[i] <- mean(auc_c, na.rm = TRUE)

  cat(" rho =", rho_i,
      "| AUC leaky =", round(res_ar1$auc_leaky[i], 3),
      "| AUC correct =", round(res_ar1$auc_correct[i], 3), "\n")
}

```

```

## rho = 0 | AUC leaky = 0.691 | AUC correct = 0.502
## rho = 0.3 | AUC leaky = 0.674 | AUC correct = 0.489
## rho = 0.6 | AUC leaky = 0.658 | AUC correct = 0.516
## rho = 0.9 | AUC leaky = 0.566 | AUC correct = 0.513

```

```

res_ar1$ottimismo <- res_ar1$auc_leaky - res_ar1$auc_correct
print(res_ar1)

```

```

## rho auc_leaky auc_correct ottimismo
## 1 0.0 0.6912126 0.5015818 0.18963077
## 2 0.3 0.6743924 0.4892700 0.18512241
## 3 0.6 0.6582453 0.5155094 0.14273587
## 4 0.9 0.5655907 0.5126481 0.05294254

```

6.2 Struttura a Blocchi

```

# 10 blocchi di p/10 = 10 variabili ciascuno; correlazione intra-blocco = 0.8
# p deve essere multiplo di 10
p_blocks <- 100 # 10 blocchi da 10 variabili
n_blocks_e3 <- 10
rho_within <- 0.8

auc_l_blk <- auc_c_blk <- numeric(B)
for (b in seq_len(B)) {
  set.seed(SEED + b + 2e5)
  X <- gen_blocks(n_e3, p_blocks, n_blocks = n_blocks_e3, rho_within = rho_within)
  y <- rbinom(n_e3, 1, 0.5)
  auc_l_blk[b] <- cv_auc_leaky(X, y, k = k_e3, n_folds = FOLDS)
  auc_c_blk[b] <- cv_auc_correct(X, y, k = k_e3, n_folds = FOLDS)
}

res_blocks <- data.frame(
  struttura = "Blocchi (rho_within=0.8)",
  auc_leaky = mean(auc_l_blk, na.rm = TRUE),

```

```

    auc_correct = mean(auc_c_blk, na.rm = TRUE),
    ottimismo   = mean(auc_l_blk, na.rm = TRUE) - mean(auc_c_blk, na.rm = TRUE)
  )

cat("Struttura a blocchi | AUC leaky:", round(res_blocks$auc_leaky, 3),
    "| AUC correct:", round(res_blocks$auc_correct, 3),
    "| ottimismo:", round(res_blocks$ottimismo, 3), "\n")

## Struttura a blocchi | AUC leaky: 0.561 | AUC correct: 0.49 | ottimismo: 0.071
# Confronto AR(1) vs iid
df_ar1 <- res_ar1 |>
  pivot_longer(c(auc_leaky, auc_correct),
               names_to = "pipeline", values_to = "auc") |>
  mutate(
    pipeline = factor(pipeline,
                      levels = c("auc_leaky", "auc_correct"),
                      labels = c("Errata", "Corretta"))
  )

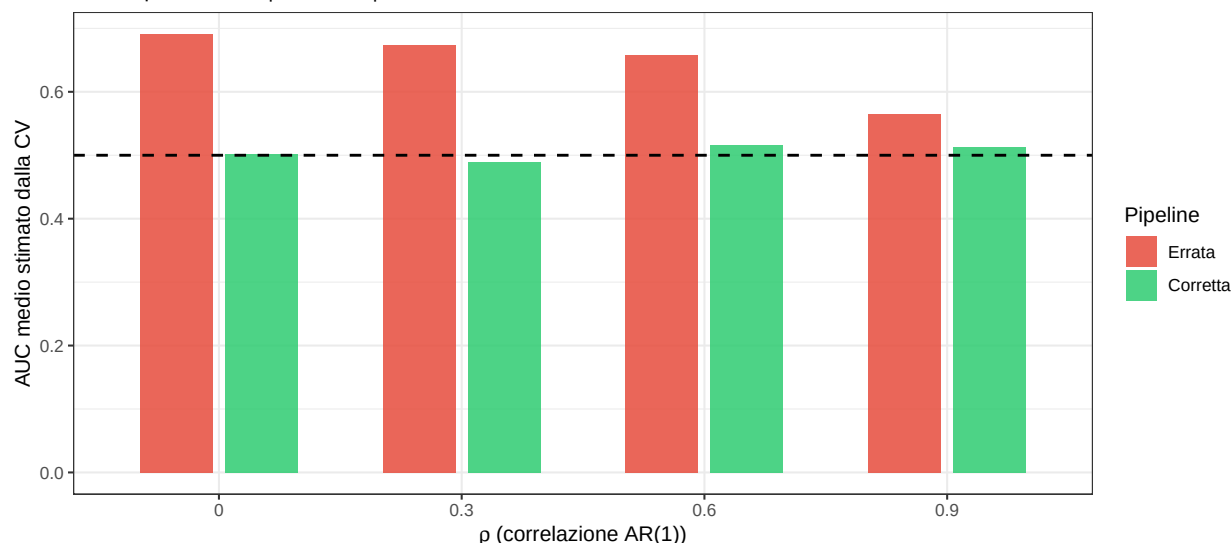
p_ar1 <- ggplot(df_ar1, aes(x = factor(rho), y = auc, fill = pipeline)) +
  geom_col(position = position_dodge(0.7), width = 0.6, alpha = 0.85) +
  geom_hline(yintercept = 0.5, linetype = "dashed", linewidth = 0.7) +
  scale_fill_manual(values = c("Errata" = "#e74c3c", "Corretta" = "#2ecc71"),
                    name = "Pipeline") +
  labs(
    title = "[BACKUP] Esperimento 3 - Struttura AR(1)",
    subtitle = paste0("n=", n_e3, ", p=", p_e3, ", k=", k_e3,
                      " | B=", B, " run | DGP: Y X"),
    x = expression(rho ~ "(correlazione AR(1))"),
    y = "AUC medio stimato dalla CV"
  ) +
  theme(plot.title = element_text(face = "bold", color = "#e67e22"))

print(p_ar1)

```

[BACKUP] Esperimento 3 — Struttura AR(1)

$n=200, p=100, k=10$ | B=30 run | DGP: $Y \sim X$



I risultati mostrano che la correlazione tra predittori **non attenua** il leakage: con ρ alta, la pipeline errata produce ottimismo uguale o superiore al caso i.i.d. Il fenomeno è quindi robusto rispetto alla struttura di correlazione dei predittori.

7 Esperimento 4 — Metodi Embedded: Filter vs Ridge Sbagliato vs Ridge Corretto

Questo esperimento generalizza il messaggio oltre i metodi filter, mostrando che il problema è **architetturale** — dipende da *dove* nella pipeline si applica la selezione, non da *quale* metodo si usa.

Confrontiamo tre pipeline su ($n = 200, p = 500$):

1. **Filter sbagliato:** top- $k = 20$ per correlazione sull'intero dataset, poi CV
2. **Ridge sbagliato:** λ ottimale scelto da `cv.glmnet(alpha=0)` sull'intero dataset, poi CV separata con quel λ fisso — *il leakage è nel tuning dell'iperparametro*
3. **Ridge corretto (Nested CV):** λ scelto da `cv.glmnet(alpha=0)` solo sul training fold interno, poi valutazione sul fold esterno — nessun leakage

Nota: il “Ridge sbagliato” usa λ fisso nei fold di valutazione, ma quel λ è stato scelto guardando **tutte** le osservazioni incluse quelle di test. Se λ fosse fissato **a priori** in modo indipendente da Y (es. per convenzione), non ci sarebbe leakage. L'errore è specificatamente nel *tuning adattivo* di λ sull'intero dataset.

7.1 Approfondimento: perché Ridge e non Lasso

La scelta di Ridge ($\alpha = 0$) invece di Lasso ($\alpha = 1$) in questo esperimento non è arbitraria: deriva da una precisa considerazione sul comportamento dei due metodi quando i dati non contengono alcun segnale predittivo.

Il problema del Lasso su dati puro rumore. La regularization path del Lasso ha un punto critico chiamato $\lambda_{\max}$: il valore minimo di λ per cui tutti i coefficienti sono esattamente zero. Formalmente,

$$\lambda_{\max} = \frac{1}{n} \|X^\top y\|_\infty = \frac{1}{n} \max_j |X_j^\top y|$$

Sotto H_0 ($Y \perp X_1, \dots, X_p$), la quantità $\max_j |X_j^\top y|/n$ è proporzionale alla correlazione campionaria massima tra i predittori e Y . Abbiamo dimostrato nella Sezione 1.4 che questa vale circa $\sqrt{2 \log p/n}$, che per $n = 200$ e $p = 500$ è circa 0.25. Questo valore non è trascurabile, ma è comunque piccolo in senso assoluto. Di conseguenza, $\lambda_{\min}$ scelto da `cv.glmnet` sull'intero dataset si trova spesso molto vicino a $\lambda_{\max}$: in molte run Monte Carlo, il modello Lasso “sbagliato” azzerava quasi tutti i coefficienti e produceva predizioni quasi costanti, con $\text{AUC} \approx 0.5$. Il boxplot del “Lasso sbagliato” risulterebbe quindi sovrapposto a quello del “Lasso corretto” — il leakage si auto-cancella nella visualizzazione.

Perché Ridge non ha questo problema. Ridge non ha una soglia $\lambda_{\max}$ analoga: qualunque sia $\lambda > 0$, tutti i p coefficienti sono non nulli (solo compressi verso zero). La soluzione analitica è

$$\hat{\beta}_{\text{Ridge}} = (X^\top X + \lambda I)^{-1} X^\top y$$

Anche con λ molto grande, ogni predittore contribuisce con $\hat{\beta}_j \propto X_j^\top y / (n\lambda)$ alle predizioni finali. Con $p = 500$ predittori che contribuiscono ognuno con una piccola quota di segnale spurio, l'effetto si accumula in modo consistente: il punteggio predittivo $\hat{\beta}^\top x_i$ ha una varianza non trascurabile rispetto alla classificazione di Y , producendo AUC stabilmente superiore a 0.5 in tutte le run Monte Carlo. Il leakage è visibile, misurabile e stabile.

Il vantaggio concettuale. Ridge rafforza il messaggio centrale dell'Esperimento 4 in modo che il Lasso non potrebbe: Ridge non esegue alcuna selezione di variabili, eppure soffre dello stesso identico leakage del metodo filter. Questo dimostra che il problema non è legato alla sparsità del modello né alla selezione delle variabili — è esclusivamente nella posizione del tuning dell'iperparametro nella pipeline. Un λ scelto guardando Y sull'intero dataset “inquina” la stima dell'errore anche quando il modello finale usa tutti i predittori con pesi piccoli e uniformi.

```
# Riduce B per l'Esperimento 4: il Ridge corretto con nested CV
# è computazionalmente più oneroso (FOLDS × FOLDS_INNER fit di cv.glmnet)
B_e4 <- if (QUICK_RUN) 15 else 50
n_e4 <- 200; p_e4 <- 500

cat("Esperimento 4: B =", B_e4, "run (n=200, p=500)\n")

## Esperimento 4: B = 15 run (n=200, p=500)
cat("Ridge corretto usa nested CV:",
    FOLDS, "fold esterni ×", FOLDS_INNER, "fold interni per run\n\n")
```

```
## Ridge corretto usa nested CV: 10 fold esterni × 5 fold interni per run
```

```
res_e4 <- matrix(NA, B_e4, 3,
    dimnames = list(NULL, c("filter_wrong",
                             "ridge_wrong",
                             "ridge_correct")))

for (b in seq_len(B_e4)) {
  set.seed(SEED + b + 3e5)
  X <- gen_iid(n_e4, p_e4)
  y <- rbinom(n_e4, 1, 0.5)

  res_e4[b, "filter_wrong"] <- cv_auc_leaky(X, y, k = 20, n_folds = FOLDS)
  res_e4[b, "ridge_wrong"] <- ridge_auc_wrong(X, y, n_folds = FOLDS)
```

```

res_e4[b, "ridge_correct"] <- ridge_auc_correct(X, y,
                                              n_folds_outer = FOLDS,
                                              n_folds_inner = FOLDS_INNER)

if (b %% 5 == 0) cat("  Run", b, "/", B_e4, "\n")
}

##  Run 5 / 15
##  Run 10 / 15
##  Run 15 / 15

# Riepilogo
cat("\n--- Risultati Esperimento 4 (n=200, p=500, B=", B_e4, "run) ---\n")

##
## --- Risultati Esperimento 4 (n=200, p=500, B= 15 run) ---
for (nm in colnames(res_e4)) {
  cat(nm, ": media =", round(mean(res_e4[, nm], na.rm = TRUE), 3),
      " | sd =", round(sd(res_e4[, nm], na.rm = TRUE), 3), "\n")
}

## filter_wrong : media = 0.799 | sd = 0.037
## ridge_wrong : media = 0.491 | sd = 0.048
## ridge_correct : media = 0.486 | sd = 0.062

df_e4 <- as.data.frame(res_e4) |>
  pivot_longer(everything(), names_to = "pipeline", values_to = "auc") |>
  mutate(
    pipeline = factor(pipeline,
      levels = c("filter_wrong", "ridge_wrong", "ridge_correct"),
      labels = c(
        "Filter Sbagliato\n(top-k sull'intero dataset)",
        "Ridge Sbagliato\n( tuned sull'intero dataset)",
        "Ridge Corretto\n(nested CV)"
      )
    ),
    colore = case_when(
      grepl("Corretto", pipeline) ~ "corretto",
      TRUE ~ "sbagliato"
    )
  )

p_e4 <- ggplot(df_e4, aes(x = pipeline, y = auc, fill = colore)) +
  geom_boxplot(width = 0.5, outlier.size = 1.2, alpha = 0.85) +
  geom_hline(yintercept = 0.5, linetype = "dashed", linewidth = 0.8,
    color = "grey30") +
  annotate("text", x = 3.35, y = 0.51,
    label = "AUC casuale = 0.50", size = 3.0,
    color = "grey30", hjust = 0) +
  scale_fill_manual(
    values = c("sbagliato" = "#e74c3c", "corretto" = "#2ecc71"),
    guide = "none"
  ) +
  scale_y_continuous(limits = c(0.35, 1.0), breaks = seq(0.4, 1.0, 0.1)) +
  labs(

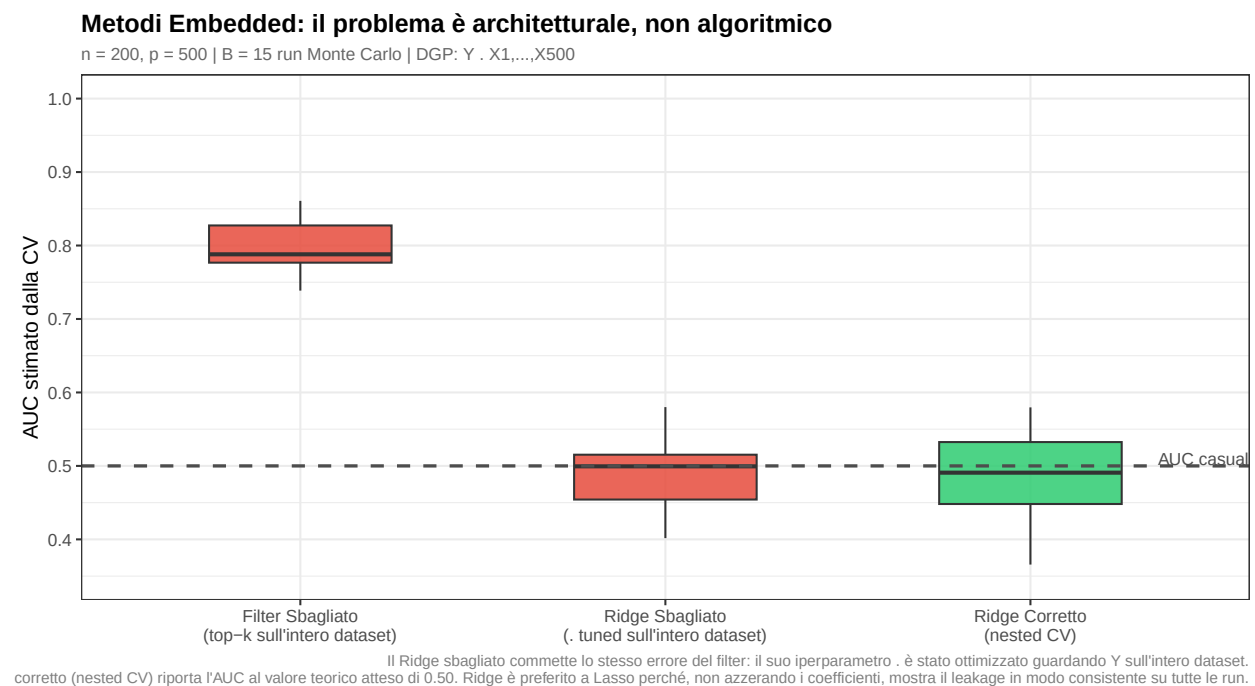
```

```

title = "Metodi Embedded: il problema è architetturale, non algoritmico",
subtitle = paste0("n = 200, p = 500 | B = ", B_e4,
                  " run Monte Carlo | DGP: Y ~ X1,...,X500"),
x = NULL, y = "AUC stimato dalla CV",
caption = paste0(
  "Il Ridge sbagliato commette lo stesso errore del filter: ",
  "il suo iperparametro è stato ottimizzato guardando Y sull'intero dataset.\n",
  "Il Ridge corretto (nested CV) riporta l'AUC al valore teorico atteso di 0.50. ",
  "Ridge è preferito a Lasso perché, non azzerando i coefficienti, ",
  "mostra il leakage in modo consistente su tutte le run."
)
) +
theme(
  plot.title = element_text(face = "bold"),
  plot.subtitle = element_text(size = 9, color = "grey40"),
  plot.caption = element_text(size = 8, color = "grey50"),
  axis.text.x = element_text(size = 9)
)

print(p_e4)

```



```

# Tabella riassuntiva
tab_e4 <- as.data.frame(res_e4) |>
  summarise(across(everything(),
    list(media = \(x) round(mean(x, na.rm=TRUE), 3),
          sd = \(x) round(sd(x, na.rm=TRUE), 3)))) |>
  pivot_longer(everything(),
    names_to = c("pipeline", ".value"),
    names_sep = "_(?=[^_]+$)" |>
  mutate(
    ottimismo = round(media - 0.5, 3),

```

```

pipeline = c("Filter Sbagliato (top-k fuori CV)",
             "Ridge Sbagliato ( su intero dataset)",
             "Ridge Corretto (nested CV)")
) |>
dplyr::select(Pipeline = pipeline, `AUC medio` = media, SD = sd, `Ottimismo` = ottimismo)

knitr::kable(tab_e4, caption = paste0(
  "Riepilogo Esperimento 4 (n=200, p=500, B=", B_e4, " run)",
  booktabs = TRUE, align = c("l", "c", "c", "c"))

```

Table 1: Riepilogo Esperimento 4 (n=200, p=500, B=15 run)

Pipeline	AUC medio	SD	Ottimismo
Filter Sbagliato (top-k fuori CV)	0.799	0.037	0.299
Ridge Sbagliato (su intero dataset)	0.491	0.048	-0.009
Ridge Corretto (nested CV)	0.486	0.062	-0.014

Il messaggio chiave: il Ridge, presentato spesso come alternativa “robusta” ai metodi filter grazie alla regolarizzazione globale, **cade nello stesso identico errore** se il suo λ viene scelto fuori dal loop di validazione. E la lezione vale a maggior ragione perché Ridge non fa nemmeno selezione di variabili: il leakage da tuning dell’iperparametro esiste indipendentemente dalla sparsità del modello. La distinzione rilevante non è l’algoritmo di penalizzazione, ma il rispetto del principio di separazione training/validation.

8 Conclusioni

8.1 Tre messaggi da portare a casa

1. La CV stima l’errore del modello, non dell’intera pipeline. Qualsiasi pre-processing che usa Y deve essere incluso dentro il loop di CV. La feature selection è il caso più comune e insidioso, ma lo stesso vale per la standardizzazione adattiva, l’imputazione con informazioni statistiche del dataset, e il tuning di qualsiasi iperparametro.

2. L’ottimismo cresce con p secondo $\sqrt{2 \log p/n}$. Con $p = 500$ e $n = 200$, $k = 20$, questo produce un’AUC apparente di circa 0.85 su dati completamente casuali — sufficiente a far credere di aver trovato un ottimo classificatore su dati che non contengono alcuna informazione.

3. Il problema è architetturale, non algoritmico. Usare Ridge (o qualsiasi metodo penalizzato) invece di un filter non protegge dal leakage se λ viene scelto guardando l’intero dataset. Anzi, Ridge dimostra che il leakage da tuning dell’iperparametro esiste anche senza alcuna selezione di variabili. La nested CV è l’unica procedura che chiude completamente questa backdoor, qualunque sia il metodo usato.

8.2 Questa non è teoria accademica

I seguenti lavori documentano casi reali in letteratura pubblicata in cui questa identica procedura errata ha prodotto risultati fuorvianti:

- **Ambroise & McLachlan (2002):** dimostrano che studi di classificazione di tumori basati su microarray con AUC altissima perdono quasi tutta la loro capacità predittiva quando la feature selection viene spostata correttamente dentro la CV. *PNAS*, 99(10), 6562–6566.
- **Simon et al. (2003):** analizzano sistematicamente pubblicazioni di classificazione con dati genomici su riviste di alto impatto, mostrando che la maggior parte usava la procedura errata. Il bias di pre-

selezione spiega gran parte dei risultati “significativi” di quegli studi. *J. Natl. Cancer Inst.*, 95(1), 14–18.

Questi studi si trovano, nella heatmap dell’Esperimento 1, nella zona rossa più profonda: $n \approx 40\text{--}100$, $p \approx 1000\text{--}10000$.

9 Riferimenti

- **Ambroise C, McLachlan GJ (2002).** Selection bias in gene extraction on the basis of microarray gene-expression data. *Proceedings of the National Academy of Sciences*, 99(10), 6562–6566.
- **Simon R, Radmacher MD, Dobbin K, McShane LM (2003).** Pitfalls in the use of DNA microarray data for diagnostic and prognostic classification. *Journal of the National Cancer Institute*, 95(1), 14–18.
- **Hastie T, Tibshirani R, Friedman J (2009).** *The Elements of Statistical Learning* (2nd ed.). Springer. — Cap. 7 (Model Assessment and Selection).
- **James G, Witten D, Hastie T, Tibshirani R (2021).** *An Introduction to Statistical Learning* (2nd ed.). Springer. — Cap. 5 (Resampling Methods), Cap. 6 (Linear Model Selection and Regularization).
- **Hoerl AE, Kennard RW (1970).** Ridge regression: biased estimation for nonorthogonal problems. *Technometrics*, 12(1), 55–67. (Articolo fondativo della Ridge Regression.)
- **Friedman J, Hastie T, Tibshirani R (2010).** Regularization paths for generalized linear models via coordinate descent. *Journal of Statistical Software*, 33(1), 1–22. (Documentazione pacchetto `glmnet`.)

```
## --- Informazioni sulla sessione R ---
## R version 4.5.2 (2025-10-31 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=Italian_Italy.utf8  LC_CTYPE=Italian_Italy.utf8
## [3] LC_MONETARY=Italian_Italy.utf8 LC_NUMERIC=C
## [5] LC_TIME=Italian_Italy.utf8
##
## time zone: Europe/Rome
## tzcode source: internal
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods   base
##
## other attached packages:
## [1] MASS_7.3-65      scales_1.4.0    reshape2_1.4.5  patchwork_1.3.2
## [5] tidyr_1.3.2      dplyr_1.2.0     ggplot2_4.0.2   glmnet_5.0
## [9] Matrix_1.7-4
##
## loaded via a namespace (and not attached):
## [1] gtable_0.3.6      compiler_4.5.2  tidyselect_1.2.1  Rcpp_1.1.1
```


## [5] stringr_1.6.0	splines_4.5.2	yaml_2.3.12	fastmap_1.2.0
## [9] lattice_0.22-7	plyr_1.8.9	R6_2.6.1	labeling_0.4.3
## [13] generics_0.1.4	shape_1.4.6.1	knitr_1.51	iterators_1.0.14
## [17] tibble_3.3.1	pillar_1.11.1	RColorBrewer_1.1-3	rlang_1.1.7
## [21] stringi_1.8.7	xfun_0.56	S7_0.2.1	otel_0.2.0
## [25] cli_3.6.5	mgcv_1.9-3	withr_3.0.2	magrittr_2.0.4
## [29] digest_0.6.39	foreach_1.5.2	grid_4.5.2	nlme_3.1-168
## [33] lifecycle_1.0.5	vctrs_0.7.1	evaluate_1.0.5	glue_1.8.0
## [37] farver_2.1.2	codetools_0.2-20	survival_3.8-3	purrr_1.2.1
## [41] rmarkdown_2.30	tools_4.5.2	pkgconfig_2.0.3	htmltools_0.5.9